

Exercise 2: Orchestration with Kubernetes using Minikube

Step 1: Install and Start Minikube Cluster

- Run: minikube delete
- Then start with Docker driver: minikube start --driver=docker
- Verify Minikube: kubectl get nodes

Step 2: Create a Simple Node.js Microservice

- Create a file named app.js with the following content:

```
const express = require('express');
```

```
const app = express();
```

```
const PORT = 8080;
```

```
app.get('/', (req, res) => {
```

```
  res.send('Hello from Kubernetes!');
```

```
});
```

```
app.listen(PORT, () => {
```

```
  console.log(`Server running on port ${PORT}`);
```

```
});
```

- Create a Dockerfile in the same directory with the following content:

```
FROM node:18
```

```
WORKDIR /app
```

```
COPY app.js .
```

```
RUN npm install express
```

EXPOSE 8080

CMD ["node", "app.js"]

Step 3: Build Docker Image Inside Minikube

- eval \$(minikube docker-env)
- docker build -t k8s-node-app .

Step 4: Create Kubernetes Deployment YAML (deployment.yaml)

apiVersion: apps/v1

kind: Deployment

metadata:

name: k8s-node-app

spec:

replicas: 2

selector:

matchLabels:

app: k8s-node-app

template:

metadata:

labels:

app: k8s-node-app

spec:

containers:

- name: node-container

image: k8s-node-app

ports:

- containerPort: 8080

Step 5: Deploy to Kubernetes and Check

- `kubectl apply -f deployment.yaml`
- `kubectl get deployments`
- `kubectl get pods`

Step 6: Expose the Microservice

- `kubectl expose deployment k8s-node-app --type=NodePort --port=8080`
- `kubectl get services`

Step 7: Test the Microservice

- Run: `minikube service k8s-node-app --url`
- Open the printed URL in your browser or run: `curl $(minikube service k8s-node-app --url)`